
# ReviewBoard

Here's a development tool that puts in place a system for single or multiple reviews of the code developed. It even offers the opportunity for self-reviews. Read on…

When I look back, it isn't that we were unaware of some of the recommended development processes, and neither did we believe they were not useful. It is just that the development infrastructure was so poor that we could not effectively implement them. It boiled down to too much effort for very small returns.

These days, it is a shame if developers do not use some of these development tools. Institutionalising some of them makes it easier, as each developer does not need to think about which ones to use. For example, these days, the default repository has become Git. For a common, centralised development repository, in contrast to a distributed environment, Subversion is a perfectly good option. However, Git can work in a centralised environment as well, and is obviously an excellent tool; so, why make an effort to select between alternatives?

Repositories can tie in with automated build systems, which will run the affected test cases and raise alerts in case an update causes regression errors. The build automation still misses issues like potential problems, security concerns or that there may be a better way to implement the code. How do we create an environment so that at least one person looks at the code?

## ReviewBoard

It is great to see that the use of ReviewBoard is increasing, including in a number of open source projects (*http://www.reviewboard.org/users/*). The original workflow of ReviewBoard was pre-commit—i.e., the code is reviewed before committing it to the repository. However, it is possible to use a post-commit workflow, where the code is submitted to the repository and then reviewed. The other very nice capability of ReviewBoard is that it can work with a fairly large number of repositories, including Git and Subversion.

Most people will find the pre-commit workflow to be the easier option. There is no ambiguity about the state of code in a repository. However, programmers cannot work on files that they have modified and are pending review. This may block them from working. In the post-commit workflow, there is the additional effort of keeping track of code in the repository that is still pending review. For repositories like Git, it is fairly easy to create a branch and merge it after the review. You may want to look at how KDE does it, at *http://techbase.kde.org/Development/Review_Board*.

As a simple illustration, consider a centralised repository like Subversion. In addition to the normal development environment, a programmer will need to install RBTools, which contain the command post-review. In a simple pre-commit scenario, a programmer will:

- Check out the code.
- Make changes.
- Use the post-review command to submit the patch to be reviewed.
- Sign into the Web interface of ReviewBoard, associate a reviewer or a group of reviewers with the code patch, and change the state from *Draft* to *Publish*.
- Modify code as per the reviewer's comments, and update the patch for review.
- Once the code is reviewed and approved, commit the changes to the repository.
- Finally, close the review request.

To a reviewer, the Web interface will show the entries needing review. If a group of people are to review the code, the review request will be shown to each member of the group. Each reviewer will continue to see the item that needs to be reviewed even if someone in the group has already reviewed it, so multiple people can continue to review and offer their comments. However, if a review request needs to be reviewed by any one person from the group, the 'Number of Reviews' field in the Web interface may be used to ignore requests that have already been reviewed but not yet been closed.

## Administering ReviewBoard

ReviewBoard is versatile, so you will need to set it up for your needs. The basic requirements will be:

- Adding users. Authentication can easily be done using LDAP, instead of a separate password for the application.
- Creating permissions for different types of users, e.g., submitters only, submitters and reviewers, reviewers with some administrative privileges, etc.
- Adding a repository.
- Optionally, creating default reviewers for various files.

ReviewBoard comes with pretty good documentation for both users and administrators. You can easily set it up and use it, even if you are a small group. It may not be a bad idea to use ReviewBoard even if you are the only developer. It gives you a chance to examine the changes you have made, and to ask yourself, 'Why did I do that?'. END

**By: Anil Seth**

The author is currently a visiting faculty member at IIT-Ropar, prior to which he was a professor at Padre Conceicao College of Engineering (PCCE) in Goa. He has managed IT and imaging solutions for Phil Corporation (Goa) and worked for Tata Burroughs/TIL. You can find him online at *http://sethanil.com/* and reach him via email at *anil@sethanil.com*.